

Software Engineering

Comprehensive Study Notes

16 Core Topics | 10% Weightage

Prepared as detailed reference notes covering all syllabus topics.

Introduction to Software Engineering

What is Software Engineering?

Software Engineering (SE) is a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software. It applies engineering principles to software creation to produce reliable, efficient, and maintainable software within cost and time constraints. The term was coined at the 1968 NATO Software Engineering Conference to address the 'software crisis' — projects were running over budget, behind schedule, and delivering unreliable software.

Core Objectives

- Produce high-quality software that meets user requirements.
- Deliver software on time and within budget.
- Ensure software is maintainable and scalable.
- Apply systematic methodologies rather than ad hoc programming.
- Manage complexity in large software systems.

Software vs. Programs

A **program** is a set of instructions. **Software** is a broader concept that includes programs, documentation, configuration data, and all associated artefacts needed to deliver a functioning system. SE focuses on the full lifecycle — not just writing code.

Key Characteristics of Good Software

- **Functionality** – does what the user needs.
- **Reliability** – performs correctly under defined conditions.
- **Efficiency** – makes good use of system resources.
- **Usability** – easy to learn and use.
- **Maintainability** – can be modified easily.
- **Portability** – can be transferred to different environments.

Key Points

- SE bridges the gap between programming and large-scale system development.
- The Software Crisis of the 1960s motivated the need for engineering discipline in software.
- Software engineering encompasses process, methods, and tools.

Software Process Models

What is a Software Process?

A software process is a structured set of activities required to develop a software system. It defines who does what, when, and how to reach the goal of building software. A process **model** (or lifecycle model) is an abstract representation of that process.

Waterfall Model

The oldest and most straightforward model. Phases flow sequentially: Requirements → Design → Implementation → Testing → Deployment → Maintenance. Each phase must be completed before the next begins. **Advantages:** simple, easy to manage. **Disadvantages:** rigid, no feedback loops, poor for changing requirements.

Incremental Model

The system is built and delivered in increments. Each increment adds functionality. Earlier increments can be used while later ones are being developed. Good when requirements are not fully understood upfront.

Spiral Model

Proposed by Barry Boehm (1988). Combines iterative development with risk analysis. Each spiral loop passes through: Planning → Risk Analysis → Engineering → Evaluation. Best suited for large, complex, high-risk projects.

V-Model (Verification & Validation)

An extension of the Waterfall model where each development stage is associated with a corresponding testing stage. The left side represents development phases; the right side represents testing phases. Emphasises early test planning.

Prototype Model

A working prototype is built early to clarify requirements. The prototype is refined through user feedback. Useful when requirements are unclear, but risks include 'prototype becoming the product.'

Key Points

- No single model fits all projects — choice depends on requirements clarity, risk, and team size.
- Waterfall suits stable requirements; Spiral suits high-risk projects.
- V-Model emphasises verification at every development stage.

Agile Software Development

The Agile Manifesto (2001)

Agile is a philosophy based on the Agile Manifesto, which values: **Individuals and interactions** over processes and tools; **Working software** over comprehensive documentation; **Customer collaboration** over contract negotiation; **Responding to change** over following a plan.

Scrum

The most popular Agile framework. Work is divided into **Sprints** (1–4 week iterations). Key roles: **Product Owner** (defines backlog), **Scrum Master** (facilitates process), **Development Team**. Key ceremonies: Sprint Planning, Daily Standup, Sprint Review, Sprint Retrospective.

Kanban

A visual workflow management method. Work items are represented on a Kanban board with columns (e.g., To Do, In Progress, Done). Work-in-progress (WIP) limits prevent overloading the team. Focuses on continuous flow rather than iterations.

Extreme Programming (XP)

XP emphasises technical excellence. Key practices include: Test-Driven Development (TDD), Pair Programming, Continuous Integration, Refactoring, Simple Design, and frequent small releases.

12 Agile Principles

- Satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development.
- Deliver working software frequently (weeks rather than months).
- Business and developers must work together daily.
- Build projects around motivated individuals.
- The most efficient method of communication is face-to-face.
- Working software is the primary measure of progress.
- Sustainable development — maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design.
- Simplicity — maximising the amount of work not done.
- Self-organising teams produce the best architectures and designs.
- Regularly reflect and adjust behaviour to become more effective.

Key Points

- Agile is iterative, collaborative, and change-friendly.
- Scrum uses fixed-length sprints; Kanban uses continuous flow.
- XP focuses on engineering practices like TDD and pair programming.

Software Requirements Engineering

What are Requirements?

A requirement is a statement of what the system should do or a constraint on how it should do it. Requirements Engineering (RE) is the process of discovering, documenting, and managing requirements. Poor requirements are the #1 cause of software project failure.

Types of Requirements

- **Functional Requirements (FR)** – what the system must do (e.g., 'The system shall allow users to log in').
- **Non-Functional Requirements (NFR)** – quality attributes (performance, security, usability, reliability).
- **Domain Requirements** – constraints from the application domain (e.g., legal regulations).

Requirements Engineering Process

- **Elicitation** – gather requirements via interviews, workshops, questionnaires, observation, prototyping.
- **Analysis** – identify conflicts, ambiguities, and incompleteness; prioritise requirements.
- **Specification** – document requirements in a Software Requirements Specification (SRS).
- **Validation** – ensure requirements reflect what stakeholders need (reviews, prototypes, test cases).
- **Management** – track changes to requirements over the project lifecycle.

Use Cases & User Stories

Use Cases describe interactions between actors and the system to achieve a goal. **User Stories** (Agile) follow the template: 'As a [role], I want [feature] so that [benefit].' Both are widely used to capture functional requirements.

Good Requirement Properties (SMART)

- Specific
- Measurable
- Achievable
- Relevant
- Testable

Key Points

- Requirements must be complete, consistent, unambiguous, and verifiable.
- Functional requirements define behaviour; NFRs define quality.
- Requirements management handles changes throughout the project.

Software Project Management

What is Software Project Management?

Software Project Management (SPM) involves planning, monitoring, and controlling software projects to deliver them on time, within budget, and to the required quality. It deals with the unique challenges of software: invisibility, complexity, and changeability.

Project Planning

Planning involves defining scope, estimating effort and cost, scheduling tasks, and allocating resources. Key techniques include Work Breakdown Structure (WBS), Gantt charts, PERT/CPM network diagrams, and Critical Path Analysis.

Effort Estimation Techniques

- **Expert Judgment** – relies on experience of senior developers.
- **Analogous Estimation** – compare with similar past projects.
- **COCOMO Model** – Constructive Cost Model uses lines of code (LOC) to estimate effort.
- **Function Point Analysis** – measures functionality delivered to the user.
- **Planning Poker** – Agile technique using consensus-based estimation.

Risk Management

Risk management involves identifying, analysing, and mitigating risks. Steps: Risk Identification → Risk Analysis (likelihood × impact) → Risk Planning → Risk Monitoring. Common risks: requirement changes, team attrition, technology problems, budget cuts.

Project Monitoring & Control

Earned Value Management (EVM) tracks project performance using: **Planned Value (PV)**, **Earned Value (EV)**, and **Actual Cost (AC)**. Key metrics: Schedule Variance ($SV = EV - PV$), Cost Variance ($CV = EV - AC$).

Key Points

- The triple constraint: Scope, Time, and Cost — changing one affects the others.
- COCOMO and Function Points are the most common estimation models.
- Risk management is a proactive, ongoing activity throughout the project.

Software Design

What is Software Design?

Software design transforms requirements into a blueprint for building the software. It involves defining the architecture, components, interfaces, and data structures. Good design leads to software that is easy to implement, test, and maintain.

Design Levels

- **High-Level / Architectural Design** – defines the overall system structure and major components.
- **Low-Level / Detailed Design** – specifies internal logic of each module (algorithms, data structures).

Design Principles

- **Abstraction** – hide unnecessary detail; focus on essentials.
- **Modularity** – divide system into manageable, independent modules.
- **Cohesion** – elements within a module should be closely related (high cohesion is good).
- **Coupling** – degree of dependence between modules (low coupling is good).
- **Separation of Concerns** – different aspects handled by different modules.
- **Information Hiding** – internal details hidden behind interfaces.
- **DRY** – Don't Repeat Yourself.
- **KISS** – Keep It Simple, Stupid.

SOLID Principles

- **Single Responsibility** – a class should have one reason to change.
- **Open/Closed** – open for extension, closed for modification.
- **Liskov Substitution** – subtypes must be substitutable for their base types.
- **Interface Segregation** – clients should not depend on interfaces they don't use.
- **Dependency Inversion** – depend on abstractions, not concretions.

Design Patterns

Reusable solutions to commonly occurring design problems. Categories: **Creational** (Singleton, Factory, Builder), **Structural** (Adapter, Decorator, Facade), **Behavioural** (Observer, Strategy, Command).

Key Points

- High cohesion and low coupling are the golden rules of good module design.
- SOLID principles guide object-oriented design for maintainability.
- Design patterns provide proven, reusable solutions to recurring problems.

Software Architecture

What is Software Architecture?

Software architecture defines the fundamental organisation of a system: its components, their relationships, and the principles guiding their design and evolution. It is the highest level of design and has the greatest impact on system quality attributes.

Architectural Styles / Patterns

- **Layered (n-tier)** – system divided into layers (Presentation, Business Logic, Data). Each layer communicates only with adjacent layers.
- **Client-Server** – clients request services from servers. Common in web applications.
- **Microservices** – application broken into small, independently deployable services. Each service has its own database and communicates via APIs.
- **Event-Driven** – components communicate by producing and consuming events. Loosely coupled.
- **Pipe and Filter** – data flows through a sequence of processing components (filters) connected by pipes. Used in compilers and data pipelines.
- **Repository / Blackboard** – components share a central data store. Common in AI and compiler systems.
- **Service-Oriented Architecture (SOA)** – services communicate via standardised protocols (e.g., SOAP/REST).

Quality Attributes (Non-Functional)

- Performance
- Scalability
- Availability
- Security
- Modifiability
- Testability

Architecture Documentation

Common views: **4+1 View Model** (Logical, Process, Development, Physical views + Use Cases). **UML diagrams** (Component, Deployment, Package diagrams) are used to document architecture.

Key Points

- Architecture decisions are hard to change later — get them right early.
- Microservices enable independent scaling and deployment of components.
- The 4+1 View Model captures architecture from multiple perspectives.

User Interface Design

Importance of UI Design

The User Interface (UI) is the point of interaction between the user and the software system. A good UI is critical for usability and user satisfaction. Poor UI design leads to user errors, frustration, and system abandonment, regardless of how well the underlying system works.

UI Design Principles (Shneiderman's 8 Golden Rules)

- Strive for consistency in design.
- Enable frequent users to use shortcuts.
- Offer informative feedback for every action.
- Design dialogues to yield closure.
- Offer simple error handling.
- Permit easy reversal of actions (undo).
- Support internal locus of control (user feels in control).
- Reduce short-term memory load.

Nielsen's 10 Usability Heuristics

- Visibility of system status.
- Match between system and the real world.
- User control and freedom.
- Consistency and standards.
- Error prevention.
- Recognition rather than recall.
- Flexibility and efficiency of use.
- Aesthetic and minimalist design.
- Help users recognise, diagnose, and recover from errors.
- Help and documentation.

UI Design Process

- **User Research** – understand users, tasks, and context.
- **Wireframing** – low-fidelity layout sketches.
- **Prototyping** – interactive mockups for user testing.
- **Usability Testing** – observe real users interacting with the interface.
- **Implementation & Iteration** – build and refine based on feedback.

Key Points

- Good UI design is user-centred — involve users from the start.
- Consistency and feedback are the most critical UI principles.
- Prototyping and usability testing are essential before final implementation.

Software Implementation & Coding

What is Software Implementation?

Implementation is the phase where the design is translated into executable source code. It involves choosing the right programming language, following coding standards, and applying best practices to produce readable, maintainable, and efficient code.

Coding Best Practices

- **Readability** – code is read more often than written; use meaningful names.
- **Commenting & Documentation** – explain 'why', not just 'what'.
- **Code Formatting** – consistent indentation and style (enforced by linters).
- **DRY Principle** – avoid code duplication; use functions and modules.
- **Error Handling** – anticipate and handle exceptions gracefully.
- **Small Functions** – each function should do one thing well.
- **Avoid Magic Numbers** – use named constants instead of literal values.

Coding Standards

Organisations adopt coding standards to ensure uniformity. Examples: Google Style Guides, PEP 8 (Python), MISRA C (safety-critical systems). Standards cover naming conventions, file structure, comment style, and error handling.

Version Control

Version control systems (VCS) like **Git** track changes to source code. Key concepts: repositories, commits, branches, merges, pull requests. Branching strategies: Git Flow, GitHub Flow, Trunk-Based Development.

Code Review

Systematic examination of code by peers before merging. Benefits: catch defects early, knowledge sharing, enforce standards. Types: formal inspection (Fagan), walkthrough, pair programming.

Key Points

- Clean, readable code reduces maintenance cost significantly.
- Git is the industry standard version control system.
- Code reviews catch up to 60% of defects before testing.

Software Testing

Purpose of Software Testing

Testing is the process of evaluating software to find defects and verify that it meets requirements. Testing can only prove the presence of bugs, not their absence (Dijkstra). The goal is to improve confidence in software quality.

Levels of Testing

- **Unit Testing** – tests individual functions or components in isolation.
- **Integration Testing** – tests interactions between integrated components.
- **System Testing** – tests the complete integrated system against requirements.
- **Acceptance Testing** – validates the system against user requirements (UAT).

Testing Techniques

- **Black-Box Testing** – tests based on input/output without knowing internal code. Techniques: Equivalence Partitioning, Boundary Value Analysis.
- **White-Box Testing** – tests internal code structure. Techniques: Statement Coverage, Branch Coverage, Path Coverage.
- **Grey-Box Testing** – partial knowledge of internal structure.

Types of Testing

- **Regression Testing** – ensure new changes don't break existing functionality.
- **Performance Testing** – load, stress, and scalability testing.
- **Security Testing** – vulnerability and penetration testing.
- **Usability Testing** – evaluate user experience.
- **Smoke Testing** – basic tests to check the build is stable.

Test-Driven Development (TDD)

TDD cycle: **Red** (write a failing test) → **Green** (write minimum code to pass) → **Refactor** (improve code without changing behaviour). Promotes design clarity and comprehensive test coverage.

Key Points

- Testing should begin early in the development lifecycle.
- 100% code coverage does not guarantee bug-free software.
- Automated testing enables faster, more reliable regression testing.

Software Maintenance & Evolution

What is Software Maintenance?

Software maintenance is the modification of a software product after delivery to correct faults, improve performance, or adapt it to a changed environment. Maintenance typically consumes 40–80% of the total software lifecycle cost.

Types of Maintenance (Swanson, 1976)

- **Corrective** – fixing bugs and defects discovered after release.
- **Adaptive** – modifying software to work in a new environment (OS upgrade, new hardware).
- **Perfective** – improving performance or adding new features based on user feedback.
- **Preventive** – restructuring code to prevent future problems (refactoring, documentation).

Lehman's Laws of Software Evolution

- **Continuing Change** – software must be continually adapted or it becomes less useful.
- **Increasing Complexity** – as software evolves, its complexity increases unless work is done to reduce it.
- **Self-Regulation** – software evolution is a self-regulating process.
- **Conservation of Organisational Stability** – average effective global activity rate is invariant.

Refactoring

Refactoring is restructuring existing code without changing its external behaviour. Common refactorings: Extract Method, Rename Variable, Replace Magic Number with Constant, Remove Dead Code. Tools: IDEs like IntelliJ, Eclipse automate many refactorings.

Legacy Systems

Legacy systems are old software systems that are still in use but are difficult to maintain. Options: continue maintenance, re-engineer, replace, retire. Business value vs. system quality matrix guides the decision.

Key Points

- Maintenance is the longest and most expensive phase of the software lifecycle.
- Refactoring improves code quality without changing behaviour.
- Legacy system management is a critical real-world challenge.

Software Quality Assurance

What is Software Quality?

Software quality is the degree to which software satisfies stated and implied needs. ISO 25010 defines quality in terms of characteristics: Functional Suitability, Performance, Compatibility, Usability, Reliability, Security, Maintainability, and Portability.

Quality Assurance vs. Quality Control

- **Quality Assurance (QA)** – process-oriented; ensures the right processes are followed to prevent defects.
- **Quality Control (QC)** – product-oriented; identifies defects in the final product through inspection and testing.

SQA Activities

- Establishing organisational quality standards and procedures.
- Monitoring project processes and products against these standards.
- Conducting formal technical reviews (FTRs) and inspections.
- Performing audits to check compliance.
- Measuring and reporting quality metrics.

Defect Prevention vs. Detection

Prevention strategies: training, process improvement, checklists, coding standards. **Detection** strategies: testing, code reviews, static analysis. Prevention is cheaper — fixing a defect in production costs 100x more than fixing it at requirements stage.

ISO 9001 & CMMI

ISO 9001 provides a framework for quality management systems applicable to any industry. **CMMI (Capability Maturity Model Integration)** has 5 maturity levels: Initial, Managed, Defined, Quantitatively Managed, Optimising. Higher maturity = more predictable quality.

Key Points

- QA is proactive (process); QC is reactive (product inspection).
- ISO 25010 is the current standard quality model for software.
- The cost of fixing defects grows exponentially the later they are found.

Software Metrics & Measurement

Why Measure Software?

'You can't manage what you can't measure' (Peter Drucker). Software metrics provide objective data to support project planning, quality assessment, process improvement, and prediction. Measurement is the basis of engineering discipline.

Types of Metrics

- **Product Metrics** – attributes of the software itself (size, complexity, defect density).
- **Process Metrics** – attributes of the development process (cycle time, defect removal efficiency).
- **Project Metrics** – used to manage projects (effort, schedule variance, cost).

Size Metrics

- **Lines of Code (LOC)** – simple but language-dependent.
- **Function Points (FP)** – measure functionality from user's perspective; language-independent.
- **Story Points** – Agile relative sizing metric.

Complexity Metrics

Cyclomatic Complexity (McCabe) – measures the number of independent paths through code. Formula: $V(G) = E - N + 2P$. Higher complexity = harder to test and maintain. Recommended threshold: $V(G) \leq 10$ per function.

Object-Oriented Metrics (CK Suite)

- **WMC** – Weighted Methods per Class.
- **DIT** – Depth of Inheritance Tree.
- **NOC** – Number of Children.
- **CBO** – Coupling Between Objects.
- **RFC** – Response For a Class.
- **LCOM** – Lack of Cohesion in Methods.

Key Points

- Function Points are language-independent and widely used in industry.
- Cyclomatic Complexity > 10 signals code that is hard to test and maintain.
- The CK suite measures OO-specific quality attributes.

Software Configuration Management

What is SCM?

Software Configuration Management (SCM) is the discipline of tracking and controlling changes to software artefacts throughout the development lifecycle. It ensures that the right version of the right artefact reaches the right person at the right time.

Key SCM Activities

- **Configuration Identification** – identifying all artefacts to be managed (code, docs, test cases). These are called Configuration Items (CIs).
- **Version Control** – tracking changes to CIs over time using tools like Git, SVN.
- **Change Control** – formal process for requesting, evaluating, approving, and implementing changes.
- **Configuration Auditing** – verifying that the built system matches its specification.
- **Configuration Status Accounting** – recording and reporting the status of CIs.

Baselines

A **baseline** is a formally reviewed and agreed-upon specification or product that serves as a reference for further development. Common baselines: Functional Baseline (requirements), Design Baseline, Product Baseline (final system).

Build & Release Management

Build Management automates compiling, linking, and packaging software (tools: Maven, Gradle, Make).

Release Management controls the deployment of software releases to different environments (Dev, Test, Staging, Production). CI/CD pipelines automate both.

CI/CD

- **Continuous Integration (CI)** – developers frequently merge code; automated builds and tests run on each commit.
- **Continuous Delivery (CD)** – software is always in a deployable state.
- **Continuous Deployment** – every passing build is automatically deployed to production.

Key Points

- SCM prevents 'it works on my machine' problems through controlled builds.
- A baseline is a snapshot that can be reproduced exactly.
- CI/CD pipelines are the modern standard for software delivery.

Software Risk Management

What is Software Risk?

A risk is a potential problem that may or may not occur in the future. Risk management identifies, analyses, and mitigates risks before they become problems. All software projects carry risk, and proactive risk management is a hallmark of mature organisations.

Risk Categories

- **Project Risks** – affect schedule or resources (e.g., staff turnover, budget cuts).
- **Product Risks** – affect quality or performance (e.g., performance bottlenecks, security flaws).
- **Business Risks** – affect the organisation (e.g., competitor launches similar product).
- **Technical Risks** – new unproven technology, integration difficulties.
- **External Risks** – regulatory changes, natural disasters, third-party dependencies.

Risk Management Process

- **Risk Identification** – brainstorming, checklists, lessons learned from past projects.
- **Risk Analysis** – assess probability and impact; calculate Risk Exposure ($RE = Probability \times Impact$).
- **Risk Prioritisation** – focus on highest-exposure risks (risk register).
- **Risk Mitigation** – strategies: Avoid, Transfer, Mitigate (reduce), Accept.
- **Risk Monitoring** – track risk indicators throughout the project.

Risk Mitigation Strategies

- **Avoidance** – change the plan to eliminate the risk.
- **Transfer** – shift the risk to a third party (insurance, outsourcing).
- **Mitigation** – take action to reduce probability or impact.
- **Acceptance** – acknowledge the risk and deal with it if it occurs (contingency plan).

Key Points

- Risk Exposure = Probability × Impact — prioritise highest exposure risks.
- A risk register documents all identified risks and their status.
- Risk management is continuous, not a one-time activity.

Software Security Engineering

Why Software Security?

Software security is the practice of building software that continues to function correctly under malicious attack. Security must be designed in from the start — it cannot be bolted on afterwards. Security breaches cause financial loss, reputation damage, and legal liability.

Core Security Principles (CIA Triad)

- **Confidentiality** – information is accessible only to authorised users.
- **Integrity** – data is accurate and has not been tampered with.
- **Availability** – systems and data are available when needed.

Common Vulnerabilities (OWASP Top 10)

- Injection (SQL Injection, Command Injection).
- Broken Authentication.
- Sensitive Data Exposure.
- XML External Entities (XXE).
- Broken Access Control.
- Security Misconfiguration.
- Cross-Site Scripting (XSS).
- Insecure Deserialization.
- Using Components with Known Vulnerabilities.
- Insufficient Logging & Monitoring.

Secure Software Development Lifecycle (SSDLC)

- **Requirements** – define security requirements and abuse cases.
- **Design** – threat modelling (STRIDE), secure design patterns.
- **Implementation** – secure coding guidelines, input validation, parameterised queries.
- **Testing** – static analysis (SAST), dynamic analysis (DAST), penetration testing.
- **Deployment** – secure configuration, least privilege, patching.
- **Maintenance** – vulnerability monitoring, incident response.

Threat Modelling (STRIDE)

- **Spoofing** – pretending to be someone else.
- **Tampering** – modifying data or code.
- **Repudiation** – denying an action.
- **Information Disclosure** – exposing data to unauthorised parties.
- **Denial of Service** – making the system unavailable.
- **Elevation of Privilege** – gaining unauthorised access.

Key Points

- Security must be built in, not bolted on — address it from requirements stage.

- The OWASP Top 10 covers the most critical web application security risks.
- STRIDE is a systematic threat modelling framework for identifying security issues.